

Spring Boot

What is Spring Boot?

- **Definition:** Spring Boot is a **framework** built on top of **Spring Framework** that **simplifies the development** of Spring-based applications by providing:
 - Auto-configuration.
 - Embedded servers (like Tomcat, Jetty).
 - Production-ready features (like Actuator for monitoring and metrics).

Purpose: Reduces boilerplate code and configuration.



Key Features of Spring Boot

1. **Auto-Configuration:** Automatically configures your application (beans) based on the classpath dependencies you include in the project, **reducing manual setup**.
2. **Embedded Server:** Comes with embedded servers like **Tomcat, Jetty, or Undertow**, allowing you to run web applications without external server setup.
3. **Starter Dependencies:** Offers "starter" packages to simplify dependency management (e.g., **spring-boot-starter-web** for web apps).
4. **Production-Ready Features:** Includes tools like **Actuator** for monitoring and health checks.

It's ideal for creating **microservices** and modern, lightweight applications with minimal configuration.

Note:-

Framework/Library	Latest Version	Notes
Java	21	Latest Long-Term Support (LTS) release
Spring Framework	6.0.x	Current stable release
Hibernate	6.2.x	Current stable release
Spring Boot	3.1.x	Current stable release

Difference between Spring and Spring boot

Aspect	Spring	Spring Boot
Type	Framework	Framework built on top of Spring
Setup	Manual configuration required	Auto-configuration provided
Dependency	Needs explicit dependency management	Built-in dependency management (Starter)
Web Server	External server setup needed	Embedded server (Tomcat/Jetty) included
Configuration	XML or Java-based	Mostly annotation-driven
Microservices	Requires manual integration	Simplified microservices development
Learning Curve	Steeper	Beginner-friendly

What is @SpringBootApplication annotation?

- It combines three annotations into one:
 - @Configuration:** Marks the class as a **configuration** class (act as a source of **@bean**)
 - @EnableAutoConfiguration:** Enables **auto-configuration** (such as setting up the **data source or Tomcat** from pom.xml)
 - @ComponentScan:** **Scans the package** for Spring components (such as classes annotated with **@Component**, **@Service**, **@Repository**, and **@Controller**, so they can be automatically registered as beans in the application context)

The flow of how a typical Spring Boot application handles an HTTP request, processes it, and responds.

- Client/External System sends an HTTP request. **Web Clients (e.g., Browsers, Mobile Applications, Postman)**The Controller Layer (**@RestController** or **@Controller**) handles the request.
- The controller invokes the Service Layer, which contains business logic.
- The service interacts with the Repository Layer to access or manipulate data.
- The Repository Layer interacts with the Database to retrieve or store data.
- Responses are sent back through the Service and Controller.
- In case of errors, the Exception Handling Layer manages exceptions and sends proper error responses.